

Software Requirements Specification

OH-MY-NEURO

로컬 Vault 기반 경량 RAG 문서 검색 시스템

소프트웨어공학개론 6조: 이윤지, 유원석, 이준, 배석현, 신현호

Instructor: 이은석

Faculty: SungKyunKwan University

Document Date: 2026-05-10

CONTENTS

OH-MY-NEURO.....	1
1. Introduction.....	4
1.1 Purpose.....	4
1.2 Scope.....	4
1.3 Definitions, Acronyms, and Abbreviations.....	4
1.4 References.....	5
1.5 Overview.....	5
2. Overall Description.....	6
2.1 Product Perspective.....	6
2.1.1 System Interfaces.....	6
2.1.2 User Interfaces.....	6
2.2 Product Functions.....	7
2.2.1 Vault 등록과 온보딩.....	7
2.2.2 증분 델타 동기화.....	7
2.2.3 문서 로드, 청킹, 임베딩 저장.....	7
2.2.4 인덱싱 상태 조회와 파일 목록 검색.....	7
2.2.5 대화형 검색과 스트리밍 응답.....	7
2.2.6 세션 기반 대화 관리.....	7
2.2.7 출처 추적.....	8
2.2.8 MCP 확장.....	8
2.2.9 런타임 설정과 인덱스 관리.....	8
2.2.10 테마와 장애 완화.....	8
2.2.11. 로컬 실행 환경 쉘 스크립트 지원.....	8
2.2.12. 에이전트 작업 컨텍스트 문서 지원.....	8
2.3 User Characteristics.....	9
2.4 Constraints.....	9
3. Specific Requirements.....	11
3.1 External Interface Requirements.....	11
3.1.1 User Interface.....	11
3.1.2 Hardware Interface.....	15
3.1.3 Software Interface.....	16
3.1.4 Communication Interface.....	17
3.2 Functional Requirements.....	18
3.2.1 Use Case Summary.....	19
3.2.2 Data Dictionary.....	23
3.2.3 Use Case Diagram.....	26
3.2.4 Data Flow Diagram.....	27
3.3 Performance Requirements.....	27
3.3.1 Static Numerical Requirements.....	27

3.3.2 Dynamic Numerical Requirements.....	28
3.4 Logical Database Requirements.....	29
3.5 Design Constraints.....	30
3.5.1 Physical Design Constraints.....	30
3.5.2 Standards Compliance.....	31
3.6 Software System Attributes.....	31
3.6.1 Reliability.....	31
3.6.2 Availability.....	32
3.6.3 Security.....	32
3.6.4 Maintainability.....	32
3.6.5 Portability.....	33
4. Supporting Information.....	33
4.1 Standard Basis.....	33
4.2 Document History.....	33

1. Introduction

1.1 Purpose

이 문서는 OH-MY-NEURO 시스템의 소프트웨어 요구사항을 명세한다. OH-MY-NEURO는 사용자가 지정한 단일 로컬 Vault 디렉토리 안의 PDF, DOCX, TXT, Markdown 문서를 벡터 데이터베이스에 증분 동기화하고, 자연어 질문에 대해 출처가 포함된 답변을 생성하는 경량 RAG 문서 검색 시스템이다.

본 문서는 사용자 관점의 기능 요구사항, 외부 인터페이스, 데이터 요구사항, 성능 요구사항, 품질 속성, 아키텍처 제약을 정리하여 구현자와 평가자가 시스템 범위와 동작 기준을 일관되게 이해하도록 하는 것을 목적으로 한다.

1.2 Scope

OH-MY-NEURO는 중소기업 조직 또는 개인 사용자가 로컬 문서 폴더를 대상으로 의미 기반 문서 검색을 수행하기 위한 데스크탑 실행형 애플리케이션이다. 사용자는 개별 파일을 반복 업로드하지 않고 하나의 Vault 디렉토리를 등록한다. 시스템은 해당 디렉토리를 스캔하여 지원 문서 형식을 찾아내고, 이전 인덱스 상태와 비교해 추가, 수정, 삭제된 파일만 ChromaDB와 동기화한다.

시스템은 FastAPI API 서버, React 기반 SPA, CLI를 제공한다. UI와 CLI는 동일한 핵심 서비스 계층을 호출하여 Vault 설정, 동기화, 상태 조회, 질문 응답, 인덱스 초기화를 수행한다. RAG 파이프라인은 LangGraph 기반 Corrective RAG로 구성되며 검색, 관련성 평가, 질의 재작성, 선택적 MCP 외부 검색, 답변 생성을 포함한다.

본 시스템은 로컬 단일 사용자 실행을 기본 범위로 한다. 다중 사용자 인증, 조직 단위 권한 관리, 클라우드 배포, OCR 기반 문서 처리, 다중 Vault 동시 관리, 장기 운영용 감사 로그는 현재 버전의 필수 범위에 포함하지 않는다.

1.3 Definitions, Acronyms, and Abbreviations

용어	정의
RAG	Retrieval-Augmented Generation. 검색 결과 기반 LLM 답변 생성 기법
LLM	Large Language Model. 대규모 언어 모델
MCP	외부 데이터 소스와 LLM 애플리케이션을 연결하는 프로토콜
LangGraph	LangChain 기반 상태 그래프 라이브러리. RAG 흐름 제어에 사용
ChromaDB	로컬 영속 저장을 지원하는 오픈소스 벡터 데이터베이스
FastAPI	HTTP API와 React SPA 정적 서빙에 사용하는 Python 웹 프레임워크
Embedding	텍스트를 벡터로 변환하는 과정

Chunking	문서를 검색에 적합한 텍스트 단위로 분할하는 과정
Vault	사용자가 검색 대상으로 지정하는 단일 로컬 디렉토리
Delta	추가, 수정, 삭제된 파일 목록
Corrective RAG	검색된 문서의 관련성을 평가하고 부족하면 질의를 재작성하여 재검색하는 RAG 방식
Document Grading	검색된 문서가 질문과 관련 있는지 평가하는 과정
Query Rewrite	검색 품질 향상을 위해 사용자 질문을 재구성하는 과정
Intent Classification	질문이 문서 검색 요청인지 일반 대화인지 분류하는 과정
Citation	답변의 근거가 된 내부 문서 또는 외부 MCP 결과 출처
Upsert	같은 source의 기존 청크를 삭제한 뒤 새 청크를 저장하는 갱신 정책

Table 1.1: Definitions, Acronyms, and Abbreviations

1.4 References

IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

LangChain Documentation: <https://python.langchain.com/docs/>

LangGraph Documentation: <https://langchain-ai.github.io/langgraph/>

ChromaDB Documentation: <https://docs.trychroma.com/>

FastAPI Documentation: <https://fastapi.tiangolo.com/>

React Documentation: <https://react.dev/>

OpenAI API Reference: <https://platform.openai.com/docs/api-reference>

Korean Law MCP: <https://github.com/pyhub-kr/mcp-server-korean-law>

1.5 Overview

본 명세서는 네 챕터로 구성된다. 첫 번째 챕터는 시스템의 목적, 범위, 용어, 참고 문서를 설명한다. 두 번째 챕터는 제품 관점, 주요 기능, 사용자 특성, 제약 조건, 의존성을 설명한다. 세 번째 챕터는 외부 인터페이스, 기능 요구사항, 성능 요구사항, 데이터 요구사항, 품질 속성, 아키텍처를 구체적으로 명세한다. 네 번째 챕터는 문서 기준과 변경 이력을 제공한다.

2. Overall Description

2.1 Product Perspective

중소규모 조직에서는 계약서, 보고서, 매뉴얼, 상담 기록 등 다양한 문서가 폴더 단위로 분산 저장된다. 기존 파일명 또는 키워드 검색은 문맥과 의미를 반영하기 어렵고, 사용자는 여러 폴더와 파일을 직접 열어보며 정보를 찾아야 한다.

OH-MY-NEURO는 이러한 문제를 해결하기 위해 하나의 로컬 Vault 디렉토리를 기준으로 문서를 인덱싱하고 자연어 질문을 처리한다. 사용자는 React 기반 웹 UI 또는 CLI에서 Vault 경로를 설정하고 SYNC를 실행한다. 시스템은 Vault 내부의 지원 파일만 스캔하고, ChromaDB에 저장된 이전 인덱스 메타데이터와 비교하여 변경된 파일만 다시 처리한다. 질문이 입력되면 LangGraph 기반 RAG 파이프라인이 관련 청크를 검색하고 출처가 포함된 답변을 생성한다.

기존 법률 SaaS나 클라우드 노트 도구와 달리, OH-MY-NEURO는 사용자 문서 원본과 벡터 인덱스를 로컬 파일 시스템에 저장한다. 따라서 사용자는 별도의 클라우드 워크스페이스에 문서를 업로드해 관리하기보다, 기존 로컬 폴더 구조를 유지한 채 문서를 검색 가능한 지식 기반으로 전환할 수 있다. 다만 임베딩 생성, 답변 생성, intent 분류, grading, rewrite 과정에서는 OpenAI API를 사용하므로 질문과 검색 컨텍스트 일부가 외부 API로 전송될 수 있다.

NotebookLM과 같은 AI 문서 질의응답 도구와의 차이는 검색 가능 여부가 아니라 문서 저장소를 관리하는 방식에 있다. NotebookLM이 사용자가 추가한 source를 중심으로 질의응답을 수행하는 도구라면, OH-MY-NEURO는 로컬 Vault 디렉토리 전체의 파일 추가, 수정, 삭제를 추적하고 변경된 문서만 증분 동기화하여 검색 인덱스를 최신 상태로 유지하는 로컬 RAG 검색 시스템이다.

2.1.1 System Interfaces

챗봇 인터페이스와 GUI 설정 패널이 포함된 웹 기반 UI 또는 CLI 커맨드는 FastAPI API 또는 서비스 계층을 통해 Vault 동기화, 질문 응답, 설정 조회, 인덱스 초기화를 수행한다. FastAPI 라우터는 도메인 로직을 직접 구현하지 않고 RAGService, index_vault_delta, VaultState, ChromaStore를 호출하는 어댑터 역할을 한다.

2.1.2 User Interfaces

사용자는 웹 기반 챗봇 인터페이스에서 명령어와 프롬프트를 입력하고, /onboarding, /chat, /vault, /settings 화면을 사용한다. CLI 환경에서도 동일한 핵심 기능을 사용할 수 있다. 두 환경 모두 인덱싱 설정과 현재 상태 확인 기능을 제공한다. 이전 대화 기록과 현재 인덱싱 상태는 사이드 패널 또는 CLI 상태 출력으로 확인할 수 있다.

2.2 Product Functions

2.2.1 Vault 등록과 온보딩

사용자는 최초 실행 시 /onboarding 화면 또는 CLI 명령 oh-my-neuro vault <path>로 Vault 디렉토리를 등록한다. 등록 상태는 ~/.oh-my-neuro/state.json에 저장된다. UI는 /api/bootstrap으로 상태를 읽고, 루트

진입 시 `onboarding_completed`가 `false`이거나 Vault 경로가 비어 있으면 `/onboarding`으로 이동하며, 완료된 상태이면 `/chat`으로 이동한다.

2.2.2 증분 델타 동기화

시스템은 Vault를 스캔하여 설정된 지원 확장자의 파일을 찾는다. 숨김 디렉토리와 설정된 제외 디렉토리는 탐색에서 제외하며, 단일 파일 크기는 기본 제한값을 초과할 경우 인덱싱하지 않고 `skipped` 목록에 기록한다. 지원되지 않는 확장자의 파일은 무시한다. 시스템은 현재 Vault의 파일 상태와 기존 인덱스 상태를 비교하여 추가, 수정, 삭제된 파일만 식별하고 반영해야 한다. 변경 감지는 `relative_path`, `mtime_ns`, `size`, `content_hash`를 기준으로 수행한다. `mtime_ns` 또는 `size`가 변경되면 `content_hash`를 다시 계산하고, `content_hash`가 기존 값과 동일하면 재인덱싱하지 않고 `last_seen_at` 등 상태 메타데이터만 갱신한다. `content_hash`가 달라진 파일만 기존 `source` 청크를 삭제한 뒤 재인덱싱한다.

2.2.3 문서 로드, 청킹, 임베딩 저장

PDF는 페이지 단위로 텍스트를 추출하고, DOCX/TXT/MD는 본문 단위로 텍스트를 추출한다. TXT/MD 인코딩은 `utf-8`, `utf-8-sig`, `cp949`, `euc-kr` 순서로 시도한다. 추출된 텍스트는 `RecursiveCharacterTextSplitter`로 청킹하고, 각 청크는 `OpenAI text-embedding-3-large` 임베딩을 통해 ChromaDB에 저장된다. 동일 `source` 재동기화 시 기존 청크를 삭제한 뒤 128개 배치로 `upsert`한다.

2.2.4 인덱싱 상태 조회와 파일 목록 검색

UI와 API는 Vault 경로, 온보딩 완료 여부, 마지막 동기화 시각, 최근 동기화 이력, 인덱싱된 파일 수, API 키 설정 여부를 조회할 수 있어야 한다. Vault 화면은 인덱싱된 파일 목록을 `source` 이름 기준으로 필터링할 수 있어야 한다.

2.2.5 대화형 검색과 스트리밍 응답

사용자가 질문을 입력하면 서비스는 빈 질문, API 키 누락, 인덱스 비어 있음 상태를 먼저 검사하고 한국어 안내 응답을 반환한다. 그 다음 `intent classifier`가 일반 대화와 문서 검색을 구분한다. 일반 대화는 RAG 그래프를 우회하고 LLM chat 응답을 생성한다. 문서 검색 질문은 ChromaDB 검색, `document grading`, `query rewrite`, MCP 라우팅, 컨텍스트 구성, 답변 생성을 거친다. UI와 CLI는 스트리밍 모드에서 `meta`, `token`, `done` 순서의 청크를 받는다.

2.2.6 세션 기반 대화 관리

브라우저 UI의 대화 세션은 서버 측 로컬 파일 저장소에 저장한다. 세션은 URL 쿼리의 `session` 값으로 선택되며, 최근 대화 목록이 사이드바에 표시된다. 대화 세션 인덱스와 각 세션 메시지는 사용자의 앱 상태 디렉토리인 `~/oh-my-neuro/chat_sessions/` 아래 JSON 파일로 보관한다. 브라우저 `localStorage`에는 다크모드 테마와 마지막 선택 `session id` 같은 UI 상태만 저장한다. 따라서 같은 로컬 사용자 계정에서 실행하는 UI에서는 브라우저를 바꾸거나 브라우저 데이터를 삭제해도 서버 측 대화 기록이 유지된다.

2.2.7 출처 추적

검색 기반 답변은 근거가 된 문서 청크를 citation으로 반환한다. citation에는 source, page 또는 location, doc_type, kind, snippet이 포함된다. MCP 결과가 사용된 경우 문서 출처와 외부 출처가 구분된다. UI는 출처를 배지 형태로 표시하고, API 응답은 전체 citation 구조를 반환한다.

2.2.8 MCP 확장

MCP 기능은 설정으로 활성화할 수 있다. 법률 키워드는 configs/mcp_servers.yaml의 law_keywords 항목으로 정의하며, 기본 예시는 법, 조항, 판례, 계약, 소송, 규정, 시행령 등이다. 질문에 law_keywords가 포함되고 MCP 클라이언트가 사용 가능한 경우 Korean Law MCP 등 외부 도구를 호출하여 컨텍스트를 보강한다. MCP 초기화 또는 호출 실패는 비치명적 오류로 처리되며, 내부 문서 검색 흐름은 계속 진행된다.

2.2.9 런타임 설정과 인덱스 관리

/settings 화면과 API는 retrieval, MCP, UI 일부 설정을 현재 프로세스에 적용한다. 이 변경은 YAML 파일에 영구 저장되는 기능이 아니라 현재 실행 중인 서버 프로세스의 설정 객체에 적용되는 런타임 변경이다. 인덱스 초기화는 CLI clear --force 또는 API /api/index/clear를 통해 수행하며, Vault 원본 파일은 삭제하지 않는다.

2.2.10 테마와 장애 완화

React UI는 브라우저 localStorage 기반 다크모드 전환을 제공한다. API 키 없음, 빈 인덱스, 빈 질문, MCP 실패, 일부 파일 파싱 실패, 프론트엔드 빌드 부재 등은 전체 시스템 중단 대신 한국어 안내 또는 폴백으로 처리해야 한다.

2.2.11. 로컬 실행 환경 셸 스크립트 지원

시스템은 로컬 실행 사용자가 최소한의 수동 설정만으로 실행 환경을 준비할 수 있도록 셸 스크립트 또는 보조 도구를 제공해야 한다. 해당 도구는 사용자의 shell 환경, 운영체제, Python 및 Node.js 사용 가능 여부, 프로젝트 의존성 설치 상태를 점검하고, 자동화 가능한 범위의 설정과 설치 절차를 수행하거나 필요한 수동 조치를 사용자에게 안내해야 한다.

2.2.12. 에이전트 작업 컨텍스트 문서 지원

시스템은 에이전트 AI 사용자가 최초 설치 시에 셸 스크립트를 직접 실행하지 않고도, 외부 에이전트 AI 및 CLI 기반 에이전트가 로컬 환경에 맞는 설치 스크립트를 열람 및 실행할 수 있도록 [A2A.md](#), [AGENTS.md](#) 와 같은 형식이 포함될 수 있으며, 프로젝트 설정과 개발 규칙을 명확히 전달하는 것을 목적으로 한다.

2.3 User Characteristics

문서 검색 사용자는 계약서, 보고서, 매뉴얼, 내부 기록을 자주 검색해야 하는 실무자이다. 기본적인 브라우저 사용 능력과 로컬 폴더 경로를 입력할 수 있는 능력이 필요하다. 문서 검색 사용자는 Python 코드를 이해할 필요 없이 UI에서 Vault 설정, SYNC 실행, 채팅 질문을 수행할 수 있어야 한다.

고급 사용자는 CLI와 설정 파일을 통해 API 키, MCP 서버, retrieval 파라미터를 조정하고 설정 및 인덱스 초기화 기능을 수행할 수 있다. 다만 이 역할은 조직 관리자, 인증 관리자, 권한 관리자가 아니며, 본 시스템의 기본 사용 시나리오는 단일 사용자의 로컬 실행이다.

2.4 Constraints

2.4.1. Hardware Interfaces

시스템은 일반 데스크탑 또는 노트북 컴퓨터에서 실행한다. 권장 사양은 4코어 이상 CPU, 8GB 이상 RAM, Vault 및 ChromaDB 인덱스를 저장할 수 있는 충분한 로컬 디스크 공간이다. 대용량 Vault를 안정적으로 처리하려면 16GB 이상 RAM과 SSD 저장장치를 권장한다. 사용자는 키보드와 마우스로 경로 입력, 질문 입력, 버튼 클릭을 수행하고, 모니터를 통해 React SPA 또는 터미널 출력을 확인한다.

2.4.2. Software Interfaces

Python 3.11 이상, Node.js 20 이상 및 npm 기반 프론트엔드 빌드 또는 개발 서버, OpenAI API 키, 선택적 MCP 서버 실행 환경이 필요하다. 주요 Python 의존성은 FastAPI, LangChain, LangGraph, ChromaDB, langchain-openai, langchain-mcp-adapters, pypdf, python-docx이다. 브라우저는 최신 Chrome, Edge, Firefox, Safari를 대상으로 한다. 운영체제는 macOS, Linux, Windows의 일반 로컬 파일 시스템을 대상으로 하며, 경로 처리에는 pathlib 기반 이식성을 확보해야 한다. 웹 브라우저는 Chrome 100 이상, Edge 100 이상, Firefox 99 이상, Safari 15 이상을 권장하며, JavaScript가 활성화되어 있어야 한다.

2.4.3. Operational Constraints

OpenAI API 키가 없으면 임베딩, 검색, 답변 생성 등 핵심 RAG 기능을 사용할 수 없다. UI는 실행되지만 API 키 누락 안내를 표시해야 한다. OCR을 기본 지원하지 않으므로 이미지 기반 스캔 PDF는 검색 가능한 텍스트를 추출할 수 없다. OCR은 ocrmypdf 등 외부 도구 연동을 통한 향후 확장 항목으로 관리한다. Vault 원본 파일은 로컬 파일 시스템에 남아 있으나, OpenAI API 호출 시 질문과 검색 컨텍스트 일부가 외부 API로 전송될 수 있다. MCP는 선택 기능이며 기본 설정에서는 비활성화되어 있다. 현재 구현은 단일 Vault 디렉토리를 전제로 한다.

2.5 Assumptions and Dependencies

본 시스템의 요구사항은 다음 가정과 외부 의존성에 영향을 받는다.

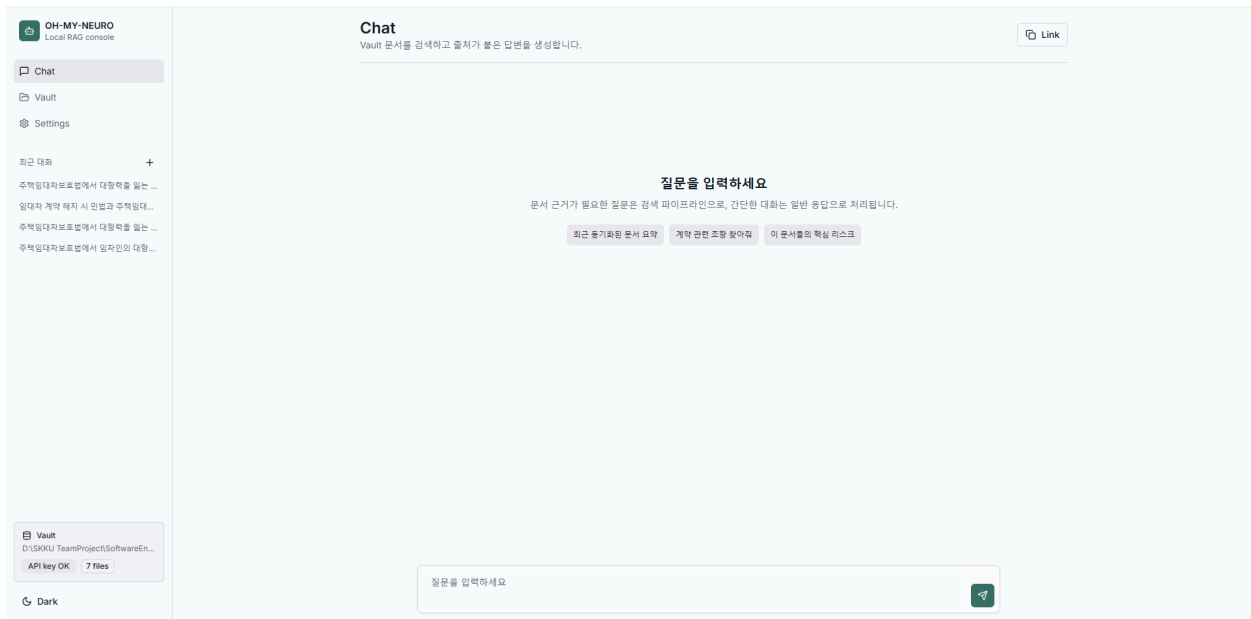
- 사용자는 검색 대상 문서를 하나의 로컬 Vault 디렉토리 아래에 보관한다고 가정한다. 사용자가 여러 독립 폴더를 동시에 검색 대상으로 사용해야 하는 요구가 생기면, 단일 Vault 기반 상태 관리와 delta sync 요구사항은 다중 Vault 관리 요구사항으로 변경되어야 한다.
- 검색 대상 문서는 PDF, DOCX, TXT, Markdown 형식이며 텍스트 추출이 가능한 문서라고 가정한다. HWP, 이미지 기반 스캔 PDF, OCR이 필 요한 문서가 주요 입력이 될 경우 문서 로더, 인덱싱 오류 처리, 성능 요구사항이 확장되어야 한다.
- 사용자는 OpenAI API에 접근 가능한 유효한 API 키와 인터넷 연결을 보유한다고 가정한다. OpenAI API 사용이 불가능하거나 내부망 전용 실행이 요구될 경우, 임베딩 모델과 LLM을 로컬 모델 또는 다른 제공자로 교체하는 요구사항이 추가되어야 한다.
- 시스템은 단일 사용자가 로컬 컴퓨터에서 실행한다고 가정한다. 여러 사용자가 동시에 접근하거나 조직 단위로 공유해야 하는 요구가 생 기면 인증, 권한 관리, 서버 측 세션 저장, 감사 로그, 배포 방식에 대한 요구사항이 추가되어야 한다.
- ChromaDB는 로컬 파일 시스템에 데이터를 저장하며, 사용자의 디스크 공간과 로컬 I/O 성능이 충분하다고 가정한다. 대용량 Vault나 네트워크 드라이브 사용이 주요 시나리오가 되면 인덱싱 성능, 저장소 관리, 동기화 안정성 요구사항을 재정의해야 한다.
- MCP 연동은 선택 기능이며 기본 동작은 내부 문서 검색만으로도 수행 가능하다고 가정한다. 외부 법령/판례 검색이 필수 기능으로 변 경될 경우, MCP 서버 가용성, 장애 처리, 응답 지연, 외부 출처 표시 방식이 필수 요구사항으로 승격되어야 한다.
- 브라우저 localStorage에는 테마와 최근 선택 session id 등 UI 상태만 저장하고, 실제 대화 세션과 메시지는 ~/.oh-my-neuro/chat_sessions/ 아래 서버 측 JSON 파일에 저장된다고 가정한다. 여러 기기 간 대화 이력 공유나 장기 보존이 필요해지면 서버 측 대화 저장소와 세션 동기화 요구사항이 추가되어야 한다.
- React production build 또는 Vite 개발 서버를 사용할 수 있는 실행 환경을 가정한다. 사용자가 단일 실행 파일 또는 설치형 데스크 탑 앱 형태를 요구할 경우 배포, UI 서빙, 업데이트 방식에 대한 요구사항이 변경되어야 한다.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interface

전체 React UI는 다음 화면으로 구성된다.



Desktop Layout	
Sidebar	Main Workspace
Navigation	Page Header
Recent sessions	Current task view
Vault/API status	Fixed chat input or data table/settings
Theme toggle	

Table 3.1: Desktop Layout

이름	React 온보딩 인터페이스
목적/내용	최초 실행 시 Vault 디렉토리를 등록하고 첫 동기화를 실행
입력 주체/출력 목적지	사용자/로컬 시스템
범위/정확도/허용 오차	존재하는 로컬 디렉토리 경로만 허용
단위	화면
시간/속도	경로 등록은 즉시, 동기화는 파일 수와 크기에 따라 비동기 처리
타 입출력과의 관계	Vault 경로 저장 API를 호출하고 동기화 진행 스트림을 수신
화면 형식 및 구성	Vault path 입력, 시작 버튼, 진행률 표시
윈도우 형식 및 구성	React SPA /onboarding
데이터 형식 및 구성	문자열 경로, NDJSON 진행 이벤트
명령 형식	경로 입력 후 시작 버튼 클릭
종료 메시지	완료 시 /chat으로 이동, 오류 시 한국어 오류 메시지 표시

Table 3.2: Onboarding 인터페이스

이름	Vault 관리 및 동기화 인터페이스
목적/내용	사용자가 현재 Vault 상태, 인덱싱된 파일 수, 마지막 동기화 시각, API Key 상태를 확인하고 인덱스를 관리한다.
입력 주체/출력 목적지	사용자 / 브라우저
범위/정확도/허용 오차	현재 등록된 Vault와 ChromaDB 인덱스 상태를 기준으로 한다.
단위	화면
시간/속도	상태 조회는 즉시 수행되며, 수동 동기화와 인덱스 초기화는 작업량에 따라 처리 시간이 달라진다.
타 입출력과의 관계	Vault 동기화, 인덱싱 파일 목록 조회, 인덱스 초기화 기능과 연동된다.
화면 형식 및 구성	Vault 경로, 인덱싱된 파일 수, 마지막 동기화 시각, API Key 상태, 파일 검색 영역, 동기화 버튼, 인덱스 초기화 버튼으로 구성된다.
윈도우 형식 및 구성	웹 브라우저 기반 React SPA의 운영 관리 화면으로 제공된다.
데이터 형식 및 구성	Vault 상태 정보, 인덱싱 파일 목록, 동기화 결과, 인덱스 초기화 결과를

	포함한다.
명령 형식	사용자가 동기화 버튼, 파일 검색 입력, 인덱스 초기화 버튼을 사용한다.
종료 메시지	동기화 완료, 인덱스 초기화 완료 또는 오류 메시지를 표시한다.

Table 3.3: Vault 관리 및 동기화 인터페이스

이름	채팅 기반 질의응답 인터페이스
목적/내용	사용자가 자연어 질문을 입력하면 시스템이 Vault 문서에서 관련 내용을 검색하고 답변을 생성한다.
입력 주체/출력 목적지	사용자 / 브라우저
범위/정확도/허용 오차	사용자는 일반적인 자연어 문장 형태로 질문을 입력할 수 있다.
단위	화면
시간/속도	질문 제출 후 즉시 응답 생성 상태를 표시하고, 가능한 경우 답변을 스트리밍 방식으로 출력한다.
타 입출력과 관계	동기화된 ChromaDB 인덱스, RAG 파이프라인, LLM 응답 생성 기능과 연동된다.
화면 형식 및 구성	대화 메시지 영역, 질문 입력창, 전송 버튼, 응답 중단 버튼, 생성 중 상태 표시 영역으로 구성된다.
윈도우 형식 및 구성	웹 브라우저 기반 React SPA의 주요 작업 화면으로 제공된다.
데이터 형식 및 구성	사용자 질문, 대화 이력, 스트리밍 응답, 최종 답변 텍스트를 포함한다.
명령 형식	사용자가 질문을 입력한 뒤 Enter 또는 전송 버튼을 클릭한다.
종료 메시지	답변 생성이 완료되면 최종 응답을 대화에 표시하고 세션에 저장한다.

Table 3.4: 채팅 기반 질의응답 인터페이스

이름	출처 및 검색 근거 확인 인터페이스
목적/내용	생성된 답변이 어떤 문서와 검색 결과를 기반으로 작성되었는지 사용자가 확인할 수 있도록 한다.
입력 주체/출력 목적지	시스템 / 사용자
범위/정확도/허용 오차	검색에 사용된 문서 출처, 페이지 또는 위치 정보, 검색 청크 수를 표시한다.

단위	화면
시간/속도	답변 생성 결과와 함께 표시된다.
타 입출력과의 관계	RAG 검색 결과, citation 정보, MCP 사용 여부와 연동된다.
화면 형식 및 구성	답변 하단 또는 주변에 출처 배지, 검색 청크 수, MCP 사용 여부를 표시한다.
윈도우 형식 및 구성	채팅 기반 질의응답 화면 내부의 보조 정보 영역으로 제공된다.
데이터 형식 및 구성	문서명, 페이지 또는 위치, 문서 유형, 검색 청크 수, 외부 MCP 출처 여부를 포함한다.
명령 형식	사용자가 답변을 확인하거나 출처 배지를 선택하여 근거 정보를 확인한다.
종료 메시지	해당 없음

Table 3.5: 출처 및 검색 근거 확인 인터페이스

이름	검색 및 MCP 설정 인터페이스
목적/내용	사용자가 검색 파라미터와 MCP 사용 여부 등 시스템 동작 설정을 조회하고 수정한다.
입력 주체/출력 목적지	사용자 / 시스템
범위/정확도/허용 오차	시스템이 허용하는 설정 항목만 수정 가능하다.
단위	화면
시간/속도	설정 저장 요청 후 현재 실행 중인 프로세스에 즉시 반영된다.
타 입출력과의 관계	RAG 검색 품질, 질의 재작성, MCP 외부 검색 여부와 연동된다.
화면 형식 및 구성	Retrieval 파라미터 입력 영역, MCP on/off 토글, API Key 상태, MCP 정보, 인덱스 초기화 영역으로 구성된다.
윈도우 형식 및 구성	웹 브라우저 기반 React SPA의 시스템 설정 화면으로 제공된다.
데이터 형식 및 구성	top_k, fetch_k, chunk_size, chunk_overlap, max_rewrites, threshold, MCP 활성화 여부를 포함한다.
명령 형식	사용자가 설정값을 수정한 뒤 저장 버튼을 클릭하거나 MCP 토글을 변경한다.
종료 메시지	설정 갱신 결과 또는 오류 메시지를 표시한다.

Table 3.6: 검색 및 MCP 설정 인터페이스

이름	공통 탐색 및 레이아웃 인터페이스
목적/내용	주요 사용자 흐름 간 이동과 반복적으로 사용되는 공통 UI 기능을 제공한다.
입력 주체/출력 목적지	사용자 / 브라우저
범위/정확도/허용 오차	온보딩 이후 주요 작업 흐름 전반에 적용된다.
단위	공통 레이아웃
시간/속도	화면 이동과 UI 상태 변경은 사용자 상호작용에 따라 즉시 반영된다.
타 입출력과 관계	채팅 세션, 최근 대화 목록, Vault 상태 요약, 화면 이동 기능과 연동된다.
화면 형식 및 구성	사이드바, 모바일 하단 네비게이션, 최근 대화 목록, 새 채팅 버튼, 현재 Vault 요약, 다크모드 토글로 구성된다.
윈도우 형식 및 구성	주요 웹 UI 화면을 감싸는 공통 React 레이아웃으로 제공된다.
데이터 형식 및 구성	라우팅 상태, 최근 대화 세션 정보, Vault 요약 상태, 테마 설정 정보를 포함한다.
명령 형식	사용자가 메뉴 선택, 새 채팅 시작, 최근 대화 선택, 다크모드 토글을 수행한다.
종료 메시지	해당 없음

Table 3.7: 공통 탐색 및 레이아웃 인터페이스

3.1.2 Hardware Interface

이름	사용자 입력 및 출력 장치
목적/내용	키보드와 마우스를 통한 사용자의 입력
입력 주체/출력 목적지	사용자/로컬 컴퓨터
범위/정확도/허용 오차	해당 없음
단위	해당 없음
시간/속도	사용자 입력 속도와 로컬 처리 속도에 의존
타 입출력과 관계	UI와 CLI 입력으로 전달

화면 형식 및 구성	React 화면 또는 터미널 텍스트 출력
윈도우 형식 및 구성	브라우저 창 또는 터미널 창
데이터 형식 및 구성	텍스트 입력, 마우스 클릭 신호
명령 형식	키보드 입력 및 마우스 클릭/CLI 명령어 입력
종료 메시지	해당 없음

Table 3.8: 하드웨어 인터페이스

3.1.3 Software Interface

이름	백엔드 API 서버
목적/내용	React SPA와 핵심 서비스 계층 사이의 어댑터
입력 주체/출력 목적지	React UI/API 클라이언트
범위/정확도/허용 오차	FastAPI 기반 REST API 엔드포인트: GET /health, GET /api/bootstrap, POST /api/vault, POST /api/vault/sync, GET /api/vault/files, POST /api/chat, POST /api/chat/stream, PATCH /api/settings, POST /api/index/clear
단위	HTTP 요청
시간/속도	로컬 요청은 즉시, RAG/동기화는 작업 시간에 의존
타 입출력과 관계	RAG 서비스, 문서 인덱싱 파이프라인, Vault 상태 관리, 벡터 스토어 호출
화면 형식 및 구성	해당 없음
윈도우 형식 및 구성	해당 없음
데이터 형식 및 구성	JSON, application/x-ndjson
명령 형식	REST-style HTTP 메서드
종료 메시지	JSON 응답 또는 NDJSON done/error 이벤트

Table 3.9: 백엔드 인터페이스

이름	웹 브라우저
목적/내용	React 인터페이스 렌더링과 사용자 상호작용

입력 주체/출력 목적지	사용자/React SPA
범위/정확도/허용 오차	Chrome 100+, Edge 100+, Firefox 99+, Safari 15+
단위	화면
시간/속도	페이지 로드와 로컬 API 응답 시간에 의존
타 입출력과의 관계	FastAPI 또는 Vite 개발 서버와 통신
화면 형식 및 구성	React SPA
윈도우 형식 및 구성	브라우저 탭
데이터 형식 및 구성	HTML, CSS, JavaScript, JSON, NDJSON
명령 형식	브라우저 이벤트
종료 메시지	해당 없음

Table 3.10: 웹 인터페이스

3.1.4 Communication Interface

이름	OpenAI API 통신
목적/내용	임베딩, intent 분류, grading, query rewrite, 답변 생성
입력 주체/출력 목적지	시스템/OpenAI API
범위/정확도/허용 오차	OpenAI API 사양과 모델별 제한에 따름
단위	HTTP 요청
시간/속도	API 응답 시간에 의존
타 입출력과의 관계	ChromaDB 검색, LangGraph 노드 실행, 스트리밍 응답 생성
데이터 형식	JSON
명령 형식	LangChain OpenAI 래퍼를 통한 API 호출
종료 메시지	오류 시 한국어 안내 메시지 또는 error 이벤트

Table 3.11: API 통신 인터페이스

이름	MCP 서버 통신
목적/내용	법률 정보 등 외부 데이터 조회

입력 주체/출력 목적지	시스템/MCP 서버
범위/정확도/허용 오차	configs/mcp_servers.yaml의 enabled 서버와 law_keywords 설정 기준
단위	MCP tool call
시간/속도	MCP 서버 응답 시간에 의존
타 입출력과 관계	RAG 그래프의 mcp 노드에서 선택적으로 호출
데이터 형식	MCP 프로토콜 메시지, 정규화된 검색 결과
명령 형식	MCP 어댑터 라이브러리를 통한 도구 호출
종료 메시지	실패 시 내부 검색만으로 진행, 사용자에게 별도 오류 미노출

Table 3.12: MCP 서버 통신 인터페이스

3.2 Functional Requirements

ID	요구사항	트리거	완료 기준
FR-01	로컬 Vault 디렉토리 등록	/onboarding 또는 vault 명령	VaultState에 경로와 온보딩 완료 저장
FR-02	Vault 변경분만 동기화	SYNC 실행	add/update/delete 델타 반영
FR-03	문서 로드, 청킹, 임베딩 저장	추가/수정 파일 발견	동일 source 기존 청크 삭제 후 upsert
FR-04	인덱싱 파일 목록 검색	/vault 파일 조회	source 기준 필터링 가능
FR-05	ChromaDB 인덱스 초기화	clear --force 또는 UI 확인	Vault 원본 파일 보존
FR-06	RAG 질의응답 수행	질문 제출	답변, citation, retrieval_count 반환
FR-07	스트리밍 채팅 응답 제공	스트리밍 질문	meta-token-done 순서 또는 error 반환
FR-08	선택적 MCP 외부 검색 보강	법률 키워드 질문	MCP 실패 시 내부 검색 유지
FR-09	retrieval, MCP, UI 런타임 설정 적용	/settings 저장	현재 프로세스 설정만 갱신

FR-10	브라우저 대화 세션 관리	채팅 UI 사용	서버 측 JSON 세션 저장 및 session 쿼리 선택
-------	---------------	----------	---------------------------------

Table 3.13: Functional Requirements Summary

FR-11 원문 문서 열기: 사용자가 citation 또는 파일 목록에서 Vault 내부 문서를 선택하면 시스템은 Vault 루트 내부의 실제 파일만 OS 기본 앱 또는 파일 위치로 열어야 한다. Vault 외부 경로, path traversal, 허용되지 않은 symlink는 거부한다.

3.2.1 Use Case Summary

Use case name	Vault Onboarding
Actor	문서 검색 사용자
Description	사용자가 로컬 Vault 디렉토리를 등록하고 첫 동기화를 실행하는 과정이다.
Normal Course	1. 사용자가 /onboarding 화면에 접속하거나 oh-my-neuro vault <path>를 실행한다. 2. 사용자가 로컬 디렉토리 경로를 입력한다. 3. 시스템이 디렉토리 존재 여부를 검증한다. 4. 시스템이 Vault 경로와 온보딩 완료 상태를 저장한다. 5. UI 온보딩에서는 /api/vault/sync를 호출해 첫 동기화를 실행한다. 6. 동기화 완료 후 사용자를 /chat으로 이동시킨다.
Precondition	시스템이 실행 중이어야 하며, 입력 경로는 존재하는 디렉토리여야 한다.
Post Condition	Vault 경로와 온보딩 완료 상태가 ~/.oh-my-neuro/state.json에 저장된다.
Assumptions	Vault 디렉토리는 사용자가 읽을 수 있는 로컬 경로이다.

Table 3.14: Use case of Vault Onboarding

Use case name	Vault Delta Sync
Actor	문서 검색 사용자
Description	Vault의 변경분만 ChromaDB 인덱스에 반영하는 과정이다.
Normal Course	1. 사용자가 /vault에서 SYNC를 누르거나 oh-my-neuro sync를

	실행한다.
	2. 시스템이 Vault를 재귀적으로 스캔한다.
	3. 지원 확장자와 최대 파일 크기 조건을 검사한다.
	4. 시스템이 기존 인덱싱된 파일 메타데이터를 조회한다
	5. 변경 감지 모듈이 추가/수정/삭제 대상 파일을 계산한다
	6. 삭제 대상 파일의 청크를 제거한다.
	7. 수정 및 추가 대상 파일을 로드, 청킹, 임베딩하여 저장한다.
	8. 진행률 이벤트와 최종 SyncResult를 UI 또는 CLI에 표시한다.
	9. 동기화 이력과 마지막 동기화 시각을 Vault 상태에 기록한다.
Precondition	Vault 경로와 OpenAI API 키가 설정되어 있어야 한다.
Post Condition	ChromaDB 인덱스가 Vault 파일 상태와 동기화된다.
Assumptions	지원 파일은 텍스트 추출 가능한 PDF/DOCX/TXT/MD 문서이다.

Table 3.15: Use case of Vault Delta Sync

Use case name	RAG Query
Actor	문서 검색 사용자
Description	사용자가 자연어 질문을 입력하면 시스템이 관련 문서를 검색하고 출처와 함께 답변하는 과정이다.
Normal Course	1. 사용자가 /chat 또는 oh-my-neuro ask에 질문을 입력한다.
	2. 시스템이 빈 질문, API 키 누락, 빈 인덱스를 검사한다.
	3. intent classifier가 일반 대화와 문서 검색 요청을 분류한다.
	4-a. 일반 대화이면 RAG 그래프를 우회하고 chat 응답을 생성한다.
	4-b. 문서 검색이면 ChromaDB에서 후보 청크를 검색한다.
	5. LangGraph grade 노드가 후보 문서의 관련성을 평가한다.
	6. 관련 문서가 없고 rewrite 횟수가 남아 있으면 query rewrite 후 재검색한다.
	7. 법률 키워드와 MCP 사용 가능 조건이 충족되면 MCP 검색을 수행한다.

	8. <code>prepare_context</code> 노드가 문서 및 외부 결과를 <code>citation</code> 과 <code>context block</code> 으로 렌더링한다.
	9. LLM이 답변을 생성하고, 시스템이 답변과 출처를 반환한다.
Precondition	질문이 입력되어야 하며, 문서 검색 요청의 경우 인덱스가 비어 있지 않아야 한다.
Post Condition	답변, 출처 목록, 검색 청크 수, MCP 사용 여부, 재작성된 질문이 반환된다
Assumptions	검색 결과가 충분하지 않을 수 있으며, 이 경우 빈 컨텍스트 안내 답변을 제공한다.

Table 3.16: Use case of RAG Query

Use case name	Streaming Chat
Actor	문서 검색 사용자
Description	답변을 토큰 단위로 브라우저 또는 CLI에 전달하는 과정이다.
Normal Course	1. 사용자가 스트리밍 질문을 입력한다. 2. 시스템이 먼저 <code>meta</code> 이벤트를 반환한다. 3. 답변 텍스트가 생성되면 <code>token</code> 이벤트를 순차적으로 반환한다. 4. 오류가 발생하면 <code>error</code> 이벤트를 반환할 수 있다. 5. 스트림 종료 시 <code>done</code> 이벤트를 반환한다.
Precondition	API 서버 또는 CLI가 실행 중이어야 한다.
Post Condition	클라이언트가 최종 응답을 조립하고 세션에 저장한다.
Assumptions	스트리밍이 불가능한 경우 비스트리밍 생성 결과로 폴백할 수 있다.

Table 3.17: Use case of Streaming Chat

Use case name	MCP Legal Search
Actor	문서 검색 사용자
Description	법률 관련 질문에 대해 내부 문서 검색 결과에 외부 MCP 검색 결과를 보강하는 과정이다.
Normal Course	1. 사용자가 법률 키워드가 포함된 질문을 입력한다.

	2. RAG 그래프가 내부 문서를 검색하고 grading한다.
	3. route 노드가 MCP 사용 가능성과 law keyword를 검사한다.
	4. MCP 클라이언트가 법률 검색 도구를 선택하여 호출한다.
	5. MCP 결과를 MCPResult로 정규화한다.
	6. 내부 문서와 외부 결과를 함께 context block에 포함한다.
Precondition	mcp.enabled가 true이고 활성 MCP 서버가 설정되어 있어야 한다.
Post Condition	답변에 MCP 외부 출처가 포함될 수 있다.
Assumptions	MCP 실패는 비치명적으로 처리하고 내부 문서 검색 흐름을 유지한다.

Table 3.18: Use case of MCP Legal Search

Use case name	Runtime Settings Update
Actor	고급 사용자
Description	실행 중인 서버 프로세스의 retrieval, MCP, UI 일부 설정을 수정하는 과정이다.
Normal Course	1. 사용자가 /settings에서 값을 수정한다.
	2. UI가 /api/settings PATCH 요청을 보낸다.
	3. 시스템은 알려진 설정 키인지 검증한다.
	4. 설정 객체를 갱신하고 RAG 서비스 또는 MCP 클라이언트를 재초기화한다.
Precondition	FastAPI 서버가 실행 중이어야 한다.
Post Condition	현재 프로세스의 후속 요청에 변경된 설정이 적용된다.
Assumptions	변경값은 YAML 파일에 영속 저장되지 않는다.

Table 3.19: Use case of Runtime Settings Update

3.2.2 Data Dictionary

VaultState

Field	Key	Constraint	Description
vault_path		String	사용자가 등록한 Vault 절대 경로
onboarding_completed		Boolean	온보딩 완료 여부
last_sync_at		Nullable ISO 8601	마지막 동기화 시각
sync_history		Max 20 entries	최근 동기화 결과 목록

Table 3.20: VaultState

SyncHistoryEntry			
Field	Key	Constraint	Description
at		ISO 8601 UTC	동기화 기록 시각
added		Integer >= 0	추가된 파일 수
updated		Integer >= 0	수정되어 재인덱싱된 파일 수
deleted		Integer >= 0	인덱스에서 삭제된 파일 수
skipped		Integer >= 0	처리되지 않고 건너뛴 파일 수
duration_s		Float >= 0	동기화 소요 시간

Table 3.21: SyncHistoryEntry

FileEntry			
Field	Key	Constraint	Description
relative_path	PK candidate	Not Null	Vault 루트 기준 상대 경로
absolute_path		Not Null	로컬 파일 절대 경로

mtime_ns	Integer	파일 수정 시각
size	Integer	파일 크기 bytes

Table 3.22: FileEntry

FileEntry는 변경 감지를 위해 content_hash, indexed_at, last_seen_at을 함께 관리한다. mtime_ns 또는 size가 변경되면 content_hash를 계산하고, content_hash가 기존 값과 같으면 재인덱싱하지 않고 last_seen_at 등 상태 메타데이터만 갱신한다.

Document Chunk			
Field	Key	Constraint	Description
chunk_id	PK	Not Null	청크 고유 식별자
content		Not Null	청크 텍스트
embedding		Not Null	청크 텍스트의 임베딩 벡터
source		Not Null	원본 문서의 Vault 상대 경로
location		Not Null	문서 내 위치 정보. 예: page=3, section=body
doc_type		Not Null	원본 문서 형식. 예: pdf, docx, txt, md
mtime_ns		Integer	파일 수정 시각
size		Integer	파일 크기 bytes
content_hash		String	파일 내용 변경 감지용 해시값

Table 3.23: Document Chunk

Citation			
Field	Key	Constraint	Description
source		Not Null	출처 문서 경로 또는 외부 출처명

location	String	문서 내 위치 정보
page	Nullable	PDF 페이지 번호
doc_type	Nullable	문서 형식
kind	document 또는 mcp	내부 문서 출처인지 외부 MCP 출처인지 구분
snippet	Nullable	참조된 텍스트 일부

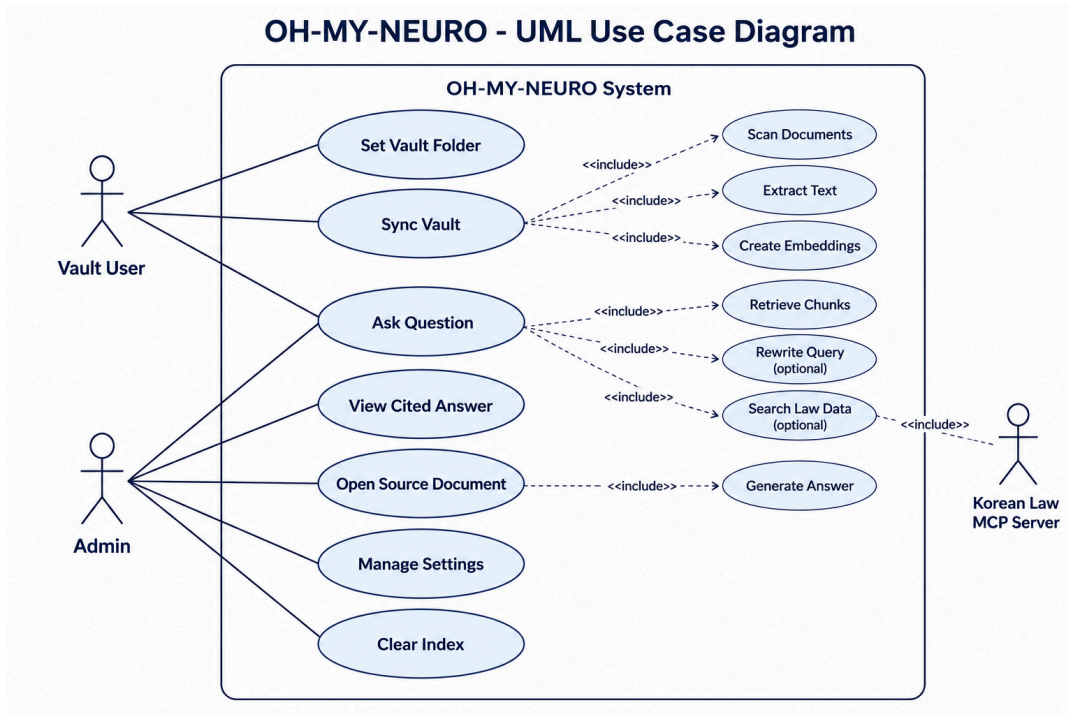
Table 3.24: Citation

ChatResponseChunk			
Field	Key	Constraint	Description
kind		meta/token/done/error	스트림 이벤트 종류
text		String	생성된 답변 토큰 또는 오류 메시지
citations		List	답변에 포함될 출처 목록
used_mcp		Boolean	MCP 결과 사용 여부
rewritten_question		Nullable	재작성된 질문
retrieval_count		Integer	검색에 사용된 관련 청크 수

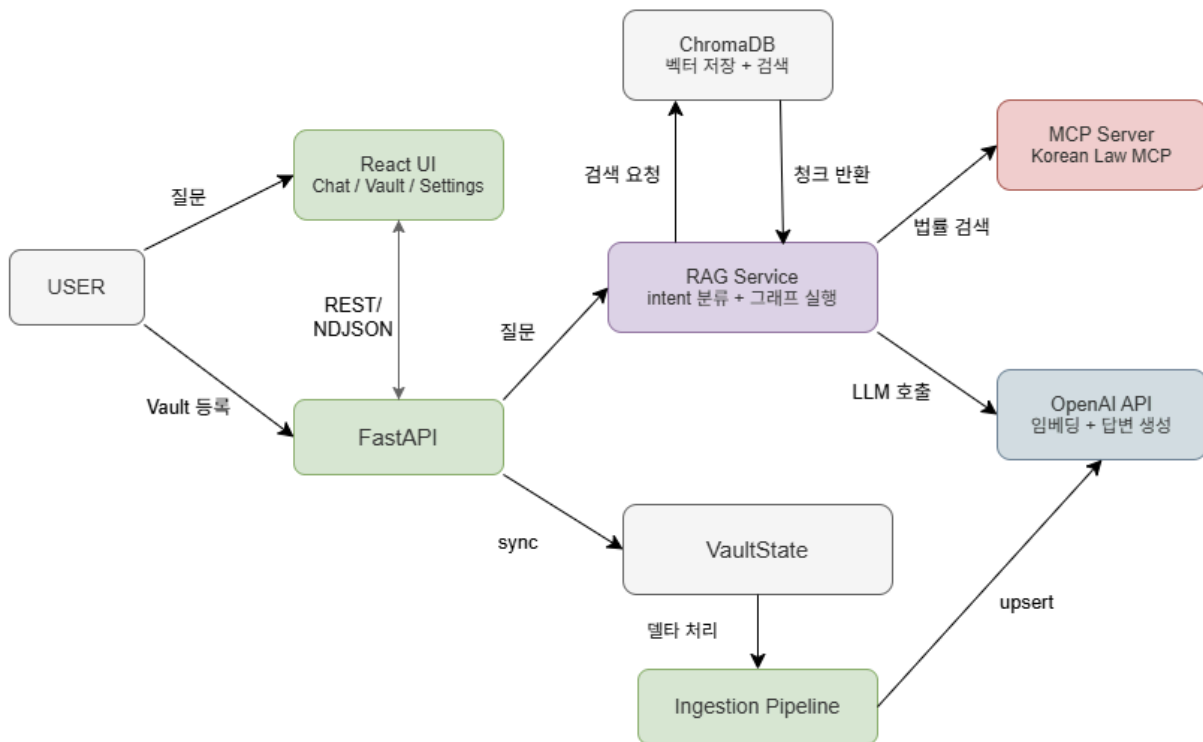
Table 3.25: ChatResponseChunk

ChatResponseChunk의 kind별 필수 필드는 다음과 같이 구분한다. meta는 retrieval_count, used_mcp, rewritten_question을 포함할 수 있다. token은 text를 필수로 포함한다. done은 citations, used_mcp, retrieval_count, rewritten_question을 포함한다. error는 text 오류 메시지를 필수로 포함하고 스트림을 종료한다.

3.2.3 Use Case Diagram



3.2.4 Data Flow Diagram



3.3 Performance Requirements

소프트웨어 또는 소프트웨어와 사람 간의 상호 작용에 적용되는 정적/동적 요구사항을 설명한다. 본 시스템은 로컬 단일 사용자 RAG 문서 검색 도구이므로 성능 요구사항은 지원 환경, Vault 규모, 동기화 진행률, 답변 지연 시간, 스트리밍 응답 순서, 장애 폴백 기준을 중심으로 정의한다.

3.3.1 Static Numerical Requirements

본 시스템은 로컬 단일 사용자 환경을 전제로 하며, 정적 수치 요구사항은 실행 환경과 처리 가능한 Vault 범위를 명시한다.

Static Numerical Requirements		
구분	요구사항	기준
사용자 수	동시 접속 사용자는 1명이다.	로컬 단일 사용자 실행
실행 환경	Python 3.11 이상과 4GB 이상 RAM의 데스크탑/노트북에서 동작한다.	Node.js/npm은 프론트엔드 개발/빌드 시 필요
파일 크기	단일 파일은 기본 50MB 이하를 지원하고 초과 파일은 인덱싱에서 제외한다.	vault.max_file_mb로 조정
Vault 규모	Vault 전체 용량은 1GB 이하를 권장한다.	문서 수와 API 처리량에 따라 처리 시간 변동
검색 설정	top_k 5, fetch_k 12, chunk_size 1000, chunk_overlap 150, max_rewrites 1, relevance_threshold 0.5를 기본값으로 둔다.	기본값은 설정 파일에서 로드, UI 변경은 현재 프로세스에만 적용
네트워크	OpenAI API 호출을 위해 1Mbps 이상 인터넷 연결을 요구한다.	임베딩/답변 생성 기능에 필요

Table 3.26: Static Numerical Requirements

3.3.2 Dynamic Numerical Requirements

동적 수치 요구사항은 실행 중 발생하는 동기화, 검색, 답변 생성, 스트리밍, 외부 MCP 호출의 응답 기준을 명시한다.

Dynamic Numerical Requirements		
구분	요구사항	평가 기준

동기화 진행	Vault 동기화 진행 상태는 UI에서 1초 이내 또는 파일 1개 처리마다 표시되어야 한다.	/api/vault/sync NDJSON progress 이벤트, 1초 이내 갱신
답변 시간	질문에 대한 첫 피드백은 즉시 표시하고, 스트리밍이 가능한 경우 첫 token은 5초 이내, 최종 답변은 기본 설정 기준 최대 90초 이내에 완료되어야 한다.	first token <= 5초, total <= 90초
즉시 피드백	질문 제출 후 UI 채팅 인터페이스는 즉시 생성 중 상태를 표시해야 한다.	전송 직후 loading 상태
스트리밍 순서	스트리밍 응답은 meta 이벤트를 먼저 전송하고 이후 token 이벤트를 순차적으로 전달해야 한다.	meta, token*, done 순서, 오류 시 error 후 종료
MCP 장애	MCP 초기화 또는 호출 실패는 전체 RAG 응답을 중단하지 않아야 하며, MCP 호출은 기본 5초 이내 timeout으로 제한한다.	mcp.timeout_s <= 5초 후 내부 문서 검색으로 폴백
파일 처리 실패	일부 파일 파싱 실패는 전체 동기화를 중단하지 않고 skipped 결과로 반환해야 한다.	SkippedFile 및 sync result에 반영

Table 3.27: Dynamic Numerical Requirements

3.4 Logical Database Requirements

데이터베이스에 저장될 정보에 대한 논리적 요구사항을 설명한다. 본 시스템은 별도 관계형 데이터베이스보다 ChromaDB 벡터 인덱스, Vault 상태 파일, YAML 설정, 서버 측 로컬 대화 세션 저장소와 브라우저 UI 상태 저장소를 조합하여 사용하므로 각 저장 영역의 정보 유형, 사용 빈도, 접근 권한, 무결성 제약을 구분한다.

Logical Database Stores			
저장 영역	정보 유형	사용 빈도/접근	무결성 제약
ChromaDB 벡터 인덱스	청크 텍스트, embedding, source metadata	검색/동기화마다 읽고 변경 파일 upsert 시 갱신	같은 source는 삭제 후 128개 배치로 upsert
VaultState	vault_path, onboarding_completed, last_sync_at, sync_history	UI/CLI 상태 조회와 동기화 완료 시 갱신	atomic write와 최근 20건 이력 제한
YAML/환경 변수 설정	AppConfig, MCPConfig, OPENAI_API_KEY, OMN_LOG_LEVEL	앱 시작과 설정 조회 시 로드	Pydantic v2 검증 및 cache reset 필요
대화 세션 저장소 / 브라우저 UI 상태	채팅 세션 메시지, 세션 인덱스, 테마 및 마지막 선택 session id	채팅 화면 사용 시 서버 파일과 브라우저 UI 상태 접근	~/oh-my-neuro/chat_sessions/ JSON 저장, localStorage에는 UI 상태만 저장
MCP 설정	서버 이름, command,	MCP 활성화 시	정의된 서버와 도구만 사용

args, law keyword	초기화/호출
-------------------	--------

Table 3.28: Logical Database Stores

ChromaDB에 저장되는 청크와 메타데이터는 다음 동기화의 변경 감지와 답변 출처 표시에 직접 사용되므로 필드 단위 무결성을 유지해야 한다.

ChromaDB Chunk Metadata			
필드	유형/제약	사용 위치	설명
chunk_id	String, PK	ChromaDB ids	청크 고유 식별자
content	Text, Not Null	documents	검색과 LLM context에 사용되는 청크 텍스트
embedding	Vector, Not Null	embedding function	text-embedding-3-large 결과 벡터
source	String, Not Null	metadata	같은 source 삭제 후 upsert하는 기준
relative_path	POSIX path, Not Null	metadata/UI	Vault 루트 기준 상대 경로
doc_type	pdf/docx/txt/md	metadata	원본 문서 형식
mtime_ns, size	Integer	delta sync	파일 변경 감지 기준
content_hash, synced_at	String/Datetime	delta sync	동일 내용 판별과 마지막 동기화 시각

Table 3.29: ChromaDB Chunk Metadata

3.5 Design Constraints

표준, 하드웨어 제약, 런타임 제약과 관련된 설계상 제한 사항을 설명한다.

3.5.1 Physical Design Constraints

시스템은 로컬 데스크탑 또는 노트북 환경에서 실행되는 것을 기본 전제로 한다.

Physical Design Constraints	
제약 항목	요구사항

실행 위치	FastAPI 서버는 기본적으로 127.0.0.1:7860에서 실행되는 로컬 애플리케이션이다.
프론트엔드 서빙	frontend/dist가 있으면 FastAPI가 React SPA를 정적으로 서빙한다.
빌드 부재	frontend/dist가 없으면 빈 화면 대신 프론트엔드 빌드가 없다는 JSON 안내를 반환한다.
개발 프록시	Vite dev server는 /api 요청을 FastAPI 서버로 프록시한다.
Vault 디렉토리	사용자 Vault 디렉토리는 자동 생성하지 않으며 존재하는 로컬 경로만 등록한다.

Table 3.30: Physical Design Constraints

3.5.2 Standards Compliance

시스템 구현은 현재 코드베이스의 Python, FastAPI, React, TypeScript, Tailwind, shadcn 스타일 구조와 프로젝트 린트 규칙을 따른다.

Standards Compliance	
표준/영역	준수 내용
Python	Python 3.11 이상과 지연 import 원칙을 따른다.
데이터 모델	Pydantic v2 API와 app.common.models의 BaseModel 기반 DTO를 우선한다.
패키징	의존성 관리는 pyproject.toml과 pip install -e .[dev] 흐름을 따른다.
린트/포맷	Ruff line-length 110, target Python 3.11, select E,F,I,B,UP,SIM을 따른다.
프론트엔드	React 19, TypeScript, Vite, Tailwind CSS, shadcn 스타일 컴포넌트 구조를 따른다.
문구/식별자	사용자 대면 문자열과 오류 메시지는 한국어, 식별자와 코드 구조는 영문으로 작성한다.

Table 3.31: Standards Compliance

3.6 Software System Attributes

3.6.1 Reliability

OH-MY-NEURO는 로컬 단일 사용자 환경에서 문서 인덱싱과 질의응답 흐름이 중단 없이 예측 가능하게 동작해야 한다. 사전 조건 오류는 RAG 그래프 실행 전에 감지하고, 일부 파일 처리 실패는 전체 동기화 실패로 전파하지 않는다.

LLM grader, query rewrite, MCP 호출, OpenAI API 호출처럼 외부 의존성이 있는 단계는 실패 시 대체 경로나 타임아웃 기준을 가져야 한다. 핵심 처리 기준은 다음 표에 함께 정리한다.

동기화, 인덱스 초기화, 질문 응답은 동시에 실행될 수 있는 작업이므로 충돌 방지 규칙을 가져야 한다. 동기화 중 추가 동기화 요청과 인덱스 초기화 요청은 409 Conflict로 거부하고, 인덱스 초기화 중 질문 응답과 동기화 요청은 수행하지 않는다. 질문 응답은 마지막으로 완료된 인덱스 상태를 기준으로 수행한다.

Software System Attribute Summary		
속성	핵심 요구사항	대표 처리 방식
Reliability	사전 조건 오류와 부분 실패의 장애 확산 방지	안내 메시지, SkippedFile, 폴백, retry/timeout
Availability	API 키 미설정 상태에서도 관리 기능 제공	비-LLM 기능 유지, atomic write, 최근 20건 이력
Security	로컬 실행과 민감 정보 보호	키 비노출, 삭제 확인, MCP allowlist
Maintainability	계층 분리와 테스트 가능한 의존성 주입	GraphDeps, Pydantic 설정, API contract 테스트
Portability	Python/Node, 브라우저 UI, POSIX 상대 경로	MCP는 선택 기능으로 유지

Table 3.32: Software System Attribute Summary

3.6.2 Availability

시스템은 FastAPI 서버와 React UI가 로컬에서 실행되는 동안 기본 관리 기능을 제공해야 한다. OpenAI API 키가 없더라도 Vault 상태 조회, 설정 조회, 화면 접근은 가능해야 하며 LLM 관련 기능만 제한한다. VaultState는 atomic write 방식으로 저장하고, 동기화 이력은 최근 20건까지 유지한다. 인덱스 초기화는 ChromaDB 컬렉션만 대상으로 하며 Vault 원본 파일은 삭제하지 않는다.

3.6.3 Security

시스템은 기본적으로 127.0.0.1에서 실행되는 로컬 애플리케이션으로 설계한다. CORS 허용 origin은 localhost와 127.0.0.1 계열로 제한한다. 0.0.0.0 등 외부 접근 가능한 host로 실행할 경우 인증 없이 Vault 등록, 파일 목록 조회, 원문 열기, 동기화, 인덱스 초기화 API를 노출하지 않아야 하며, 인증, 접근 제어, 네트워크 보호는 별도 운영 요구사항으로 다룬다. Vault 경로 처리 시 path traversal, Vault 외부 경로, 허용되지 않은 symlink 접근은 거부한다.

OpenAI API 키는 .env 또는 환경 변수로 주입하고 화면, API 응답, 로그에 원문을 노출하지 않는다. 질문과 검색 컨텍스트 일부가 외부 API로 전송될 수 있다는 점은 사용자에게 명시한다.

인덱스 삭제와 MCP 연동은 사용자가 명시적으로 선택한 경우에만 수행되어야 한다. 삭제는 확인 절차를 거치고, MCP는 설정 파일에 정의된 서버와 도구만 호출한다.

3.6.4 Maintainability

시스템은 UI, API, 서비스, RAG 그래프, 인제스트, 스토리지, 설정, MCP 계층을 분리한다. FastAPI 라우터는 얇은 adapter로 유지하고, 도메인 로직은 전용 모듈에 위임한다. RAG 그래프는 GraphDeps Protocol을 통해 테스트 대역을 주입할 수 있어야 한다. 설정은 Pydantic v2 모델로 검증하고, 새 문서 형식이나 API/스트리밍 변경 시 관련 테스트를 함께 갱신한다. 코드는 Python 3.11, Ruff 규칙, 자연 import 원칙을 따른다. 사용자 대면 문자열은 한국어, 식별자와 코드 구조는 영문을 사용한다.

3.6.5 Portability

OH-MY-NEURO는 Python 3.11 이상과 Node.js/npm 환경이 있는 일반 데스크탑 또는 노트북에서 실행 가능해야 한다. 특정 운영 체제 전용 GUI 프레임워크 대신 브라우저 기반 React UI를 사용한다. 경로 처리는 pathlib.Path, expanduser, resolve를 사용하고, Vault 내부 경로는 POSIX 스타일 상대 경로로 관리한다. 기본 상태 경로와 ChromaDB 경로는 로컬 파일 시스템 정책에 맞게 확인 가능해야 한다. 프로덕션 UI는 frontend/dist 빌드 결과에 의존하고, 개발 환경은 Vite dev server와 FastAPI 조합을 지원한다. MCP가 설치되지 않은 환경에서도 기본 문서 검색 기능은 사용할 수 있어야 한다.

4. Supporting Information

4.1 Standard Basis

본 문서는 IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998의 구성을 참고하여 작성되었다.

4.2 Document History

Date	Version	Description	Writer
2026-05-03	V1.00	Product Perspective, Product Functions	이윤지
2026-05-03	V1.00	User Characteristics, Constraints, Assumptions and Dependencies	유원석
2026-05-03	V1.00	External Interface Requirements, Functional Requirements	이준
2026-05-03	V1.00	Performance Requirements, Logical Database Requirements, Design Constraints	신현호
2026-05-03	V1.00	Software System Characteristics, System	배석현

Architecture, System Evolution			
2026-05-08	V2.00	코드베이스 기준 SRS 형식 정리 및 표 복구	이윤지, 유원석, 이준, 신현호, 배석현

Table 4.1: Document History